



Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Arquitectura del Computador

Análisis de accesos a memoria en algoritmos de ordenamiento

Informe Técnico

Alumno:
José Quintana

Profesor:
José Canache

Barbula, abril 2026

Índice

1. El problema	2
2. Técnicas computacionales para su solución	2
2.1. Arquitectura de Clases	2
2.2. Estructuras de Datos	3
2.3. Algoritmos y patrones de diseño	3
3. Comparación de resultados	4
3.1. Prueba con datos sintéticos	4
3.2. Prueba con distribuciones aleatorias	4
3.3. Prueba con datos reales	4
3.4. Análisis de las pruebas realizadas	5
4. Estrategias de optimización para la jerarquía de memoria	5
4.1. Optimización del ordenamiento para la reducción de fallos	5
4.2. Ajuste del tamaño de bloque y minimización de E/S	6

1. El problema

El rendimiento de los algoritmos de ordenamiento tradicionalmente se evalúa mediante su complejidad temporal algorítmica. Sin embargo, en sistemas modernos, el factor limitante no es siempre la velocidad de la CPU, sino el cuello de botella en la jerarquía de memoria.

El problema central de este proyecto radica en la ineficiencia que surge cuando algoritmos como QuickSort y MergeSort procesan grandes volúmenes de datos que exceden la capacidad de la memoria caché. Al acceder a datos de manera no lineal o saltar entre direcciones de memoria distantes, se generan fallos de caché constantes. Esto obliga al sistema a buscar datos en la memoria principal o el disco, incrementando drásticamente la latencia y el tiempo total de ejecución.

El objetivo es cuantificar este impacto mediante la medición de los accesos a caché y evaluar cómo la estructura de los algoritmos afecta la localidad de referencia. Además, se busca identificar estrategias de optimización, como el ajuste del tamaño de bloque, para minimizar las operaciones de entrada/salida (E/S) y mejorar el rendimiento en conjuntos de datos de gran escala.

2. Técnicas computacionales para su solución

Para llevar a cabo la simulación y el análisis de los accesos a memoria de forma transparente, el proyecto se fundamenta en un diseño orientado a objetos que integra diversas estructuras de datos y algoritmos. A continuación, se detallan las técnicas implementadas en los distintos módulos del código fuente:

2.1. Arquitectura de Clases

La modularidad del proyecto se divide en tres componentes principales:

- **CacheSimulator:** Clase responsable de mantener el estado de la jerarquía de memoria. Encapsula las dimensiones de la caché, gestiona el vector de líneas físicas y expone métodos para contabilizar el rendimiento mediante el cálculo dinámico del *Miss Rate* y *Hit Rate*.
- **DatasetGenerator:** Espacio de nombres que aísla la lógica de generación de cargas de trabajo. Además de proveer datos sintéticos y aleatorios, incluye rutinas de extracción de datos reales desde archivos .csv o .txt, incorporando un robusto manejo de excepciones para ignorar encabezados o datos anómalos sin detener la ejecución.
- **TrackedArray y ProxyElement:** Conjunto de clases basadas en plantillas que funcionan como una envoltura del vector de datos. La clase interna **ProxyElement** mantiene referencias directas tanto al valor real como al simulador, actuando como el puente de comunicación que dispara los eventos de acceso a memoria cada vez que un algoritmo de ordenamiento interactúa con los datos.

2.2. Estructuras de Datos

El almacenamiento y la representación del estado de la caché se manejan a través de las siguientes estructuras:

- **Línea de caché (`struct CacheLine`):** Es la estructura que modela una línea física dentro del simulador. Contiene un valor booleano (`valid`) que indica si la línea posee información real (evitando la lectura de valores residuales en memoria) y un entero sin signo de 64 bits (`tag`) que almacena el identificador único del bloque de memoria alojado.
- **Vectores dinámicos (`std::vector`):** Utilizados como la estructura principal para garantizar la contigüidad en memoria.
 - Se emplea un `std::vector<CacheLine>` predimensionado para representar la totalidad de las líneas de la memoria caché simulada.
 - Se utiliza la plantilla genérica `std::vector<T>` (y específicamente `std::vector<int>`) para almacenar las colecciones de datos correspondientes a las cargas de trabajo que procesan los algoritmos de ordenamiento.

2.3. Algoritmos y patrones de diseño

La lógica central de la simulación y la generación de datos se rige por los siguientes algoritmos y metodologías:

- **Algoritmo de mapeo directo:** Implementado para resolver las direcciones de memoria simuladas. Dada una dirección de 64 bits, el algoritmo realiza aritmética de enteros para descomponerla:
 1. Calcula la dirección del bloque eliminando el desplazamiento: `blockAddress = address / blockSize`.
 2. Determina la línea física en la caché mediante el módulo: `index = blockAddress % numLines`.
 3. Extrae la etiqueta del bloque: `tag = blockAddress / numLines`.

Con estos datos, evalúa si existe coincidencia en el estado de la línea para contabilizar un acierto o gestionar un reemplazo por fallo.

- **Algoritmos estocásticos y de llenado secuencial:** Para la generación de escenarios de prueba, el sistema automatiza la creación de datos mediante algoritmos estándar de C++:
 - Para datos sintéticos se usa `std::iota` (llenado incremental) y `std::reverse` para simular los mejores y peores casos algorítmicos.
 - Para datos aleatorios, se implementa el motor de números pseudoaleatorios **Mersenne Twister** (`std::mt19937`) inicializado con una semilla de hardware. Mediante `std::uniform_int_distribution` y `std::normal_distribution`, se generan escenarios de incertidumbre total y campanas de Gauss (distribución de datos donde se agrupan en torno a un valor central).

- **Patrón de diseño proxy e intercepción de accesos:** Constituye el núcleo de la monitorización no intrusiva. A través de la sobrecarga de operadores (`operator[]`, `operator=` y operadores de conversión de tipo), se interceptan de manera transparente las lecturas y escrituras en el arreglo. Al acceder a una posición, el algoritmo calcula su dirección simulada con la fórmula `baseAddress + (index * sizeof(T))` y notifica al simulador antes de devolver el valor real.

3. Comparación de resultados

3.1. Prueba con datos sintéticos

En las pruebas con secuencias ordenadas e inversas, QuickSort presenta un Hit Rate excepcional (99.8 % - 99.9 %). Esto se debe a que el algoritmo trabaja sobre particiones contiguas, aprovechando al máximo la localidad espacial. Sin embargo, el volumen de accesos totales es masivo (hasta 200 millones para solo 10,000 elementos), lo que indica una degradación a complejidad $O(n^2)$ debido a una elección de pivote poco óptima para datos ya ordenados.

MergeSort mantiene un volumen de accesos muy bajo y estable, confirmando su complejidad $O(n \log(n))$. No obstante, su Hit Rate es menor comparado con QuickSort. Esto ocurre porque MergeSort sufre de una baja localidad temporal, ya que requiere un arreglo auxiliar, lo que duplica el espacio de direccionamiento activo y genera más fallos por la necesidad de saltar entre el arreglo original y el temporal durante la fase de mezcla.

3.2. Prueba con distribuciones aleatorias

Cuando los datos no tienen un orden predefinido, ambos algoritmos muestran un comportamiento más equilibrado. QuickSort vuelve a liderar en eficiencia de caché con mayor promedio de aciertos. Al particionar el arreglo in-place, el procesador encuentra los datos necesarios en las líneas de caché previamente cargadas.

Aunque MergeSort tiene una tasa de fallos consistentemente superior, su rendimiento no se ve afectado por la distribución de los datos, lo que lo hace más predecible en sistemas donde la latencia de memoria es constante.

3.3. Prueba con datos reales

El dataset usado trata sobre precios de acciones mundiales con 310,122 elementos, representados como numeros flotantes. La prueba revela la realidad del rendimiento en sistemas de gran escala. El numero de accesos en Quicksort se dispara a 1,944 millones, con 33 millones de fallos de caché. A pesar de mantener un Hit Rate del 98.2 %, la penalización acumulada por el volumen de fallos anula cualquier ventaja de localidad. Mientras, MergeSort solo accede 17.7 millones y una tasa de fallos del 2.3 %, MergeSort demuestra ser más eficiente en el uso total del bus de memoria para datasets grandes y no controlados.

3.4. Análisis de las pruebas realizadas

Tras analizar los datos recolectados por el simulador, podemos concluir que QuickSort demuestra sistemáticamente un Hit Rate superior. Esto confirma que los algoritmos que operan sobre el mismo segmento de memoria minimizan la fragmentación de la caché y aprovechan mejor la carga de bloques de 64 bytes (localidad espacial), ya que los elementos adyacentes al pivote suelen estar ya presentes en la caché.

Por otro lado, MergeSort, a pesar de ser algorítmicamente estable, tiene un alto costo en memoria. El uso de un buffer temporal no solo aumenta el uso de RAM, sino que provoca fallos de conflicto en la caché al competir por las mismas líneas de mapeo directo, lo que explica por qué su tasa de fallos siempre es aproximadamente 1 % más alta que la de QuickSort. Además, en una caché de mapeo directo, si el arreglo original y el auxiliar están separados en memoria por una distancia que es múltiplo del tamaño de la caché, ocurrirán fallos por conflicto constantes al competir por el mismo índice. Sin embargo, en una cache asociativa por conjuntos, los resultados favorecen a MergeSort, ya que permite leer del arreglo original y escribir en el auxiliar sin que se expulsen mutuamente.

Un punto fundamental es que un Hit Rate alto no garantiza rapidez si el algoritmo no es eficiente. El caso de los datos ordenados en QuickSort es el ejemplo perfecto: a pesar de tener un 99.9% de aciertos, el exceso de operaciones innecesarias satura el sistema. Un algoritmo con un Hit Rate ligeramente inferior pero con menor complejidad como MergeSort, resulta en menos tráfico total hacia la memoria principal.

4. Estrategias de optimización para la jerarquía de memoria

Tras el análisis de los resultados obtenidos, se identifican dos vías críticas para mejorar el rendimiento: la modificación del flujo algorítmico para adaptarse a la capacidad de la caché y el ajuste de los parámetros del hardware simulado para optimizar el ancho de banda del bus.

4.1. Optimización del ordenamiento para la reducción de fallos

El análisis demostró que incluso con un *Hit Rate* cercano al 100 %, el volumen masivo de accesos en QuickSort ante datos ordenados degrada el sistema. Para mitigar esto, se proponen las siguientes optimizaciones:

- **Algoritmos híbridos:** Una estrategia eficaz consiste en implementar un umbral de recursión. Cuando una partición de QuickSort o MergeSort alcanza un tamaño lo suficientemente pequeño como para caber íntegramente en la caché L1, el algoritmo debe conmutar a InsertionSort. Esto elimina la sobrecarga de llamadas recursivas y aprovecha la *localidad temporal*, manteniendo los datos calientes en los registros y niveles superiores de la caché durante el ordenamiento final de la pequeña sublista.
- **Pivote aleatorio o media de tres:** Para evitar el peor caso de QuickSort observado en las pruebas sintéticas, el uso de un pivote aleatorio garantiza que la partición

sea balanceada. Esto reduce la profundidad del árbol de recursión y, por ende, el número total de comparaciones (accesos a memoria), atacando el problema desde la raíz de la complejidad algorítmica.

4.2. Ajuste del tamaño de bloque y minimización de E/S

El tamaño del bloque de caché es la unidad mínima de transferencia entre la memoria principal y el procesador. Su ajuste impacta directamente en la eficiencia del bus de datos y en las operaciones de Entrada/Salida (E/S).

Al incrementar el tamaño del bloque (por ejemplo, de 64 a 128 bytes), cada fallo de caché trae consigo más elementos contiguos. En algoritmos como QuickSort, que operan sobre segmentos de memoria adyacentes, esto reduce la frecuencia de los fallos, ya que la probabilidad de encontrar el siguiente elemento en el mismo bloque cargado aumenta.

No obstante, un bloque excesivamente grande puede provocar contaminación de caché, al cargar datos que el algoritmo nunca llegará a procesar antes de que sean expulsados. Además, bloques más grandes incrementan la penalización por fallo, ya que el bus de datos requiere más ciclos de reloj para transferir el bloque completo, lo que puede saturar el bus y aumentar la latencia total en datasets de gran escala como el de precios de acciones.